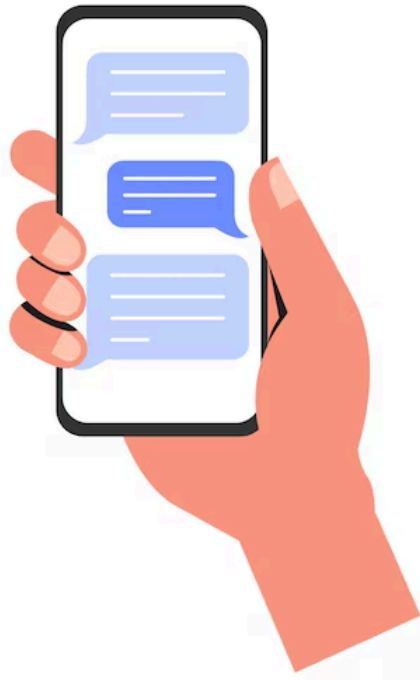


# **Sistemas Distribuidos**

## **Práctica Final – Servicio de envío de mensajes**

**Grado en Ingeniería Informática – Curso 2025/26**



**Grupo 82**

**Carmen Peláez Martín - 100522094@alumnos.uc3m.es**

**Ana Sanz del Collado - 100522234@alumnos.uc3m.es**

## 1. Índice de contenidos

|                                                    |    |
|----------------------------------------------------|----|
| 2. Introducción                                    | 1  |
| 3. Descripción del código                          | 1  |
| - Servidor Central (Lenguaje C)                    | 1  |
| - Cliente de mensajería (Python)                   | 2  |
| - Transferencia de Ficheros Peer-to-Peer (Sockets) | 2  |
| - Servicio Web (Python)                            | 2  |
| - Servicio de Registro de Operaciones (RPC)        | 2  |
| 4. Compilación y ejecución                         | 3  |
| 5. Batería de pruebas y resultados                 | 3  |
| 6. Conclusiones y problemas encontrados            | 11 |

## 2. Introducción

El objetivo de esta práctica es conocer y practicar los principales conceptos relacionados con el diseño e implementación de una aplicación distribuida. El sistema desarrollado es un servicio de notificación de mensajes entre usuarios conectados a Internet, con una funcionalidad similar a WhatsApp. El proyecto combina diferentes tecnologías para el desarrollo de aplicaciones distribuidas

## 3. Descripción del código

El desarrollo de esta práctica se ha dividido en varios componentes distribuidos que interactúan entre sí utilizando diferentes tecnologías (Sockets, Servicios Web y RPC). A continuación, se detallan las decisiones de diseño y las funciones principales de cada módulo:

### - Servidor Central (Lenguaje C)

El servidor principal es concurrente y multihilo, diseñado para proporcionar el servicio de comunicación entre los clientes registrados. Gestiona una lista enlazada en memoria con todos los usuarios registrados, almacenando su estado de conexión, dirección IP, puerto de escucha, y una cola de mensajes pendientes

Al utilizar hilos POSIX (pthreads), el acceso a esta lista compartida está protegido mediante un mutex global para evitar condiciones de carrera

El servidor utiliza sockets TCP orientados a conexión y asigna a cada cliente un hilo desacoplado (detached) que procesa la petición, invoca al servicio RPC y cierra el socket tras responder

- Cliente de mensajería (Python)

El cliente es una aplicación de consola que permite interactuar con el sistema. Su arquitectura es multihilo: el hilo principal maneja la entrada estándar (shell del usuario) y se encarga de enviar las peticiones al servidor central de forma síncrona

Al realizar la operación de conexión (CONNECT), el cliente busca un puerto libre y crea un hilo secundario en modo demonio. Este hilo levanta un socket servidor (listener) que se queda a la escucha para recibir de forma asíncrona los mensajes provenientes del servidor central o peticiones directas de transferencia de ficheros

- Transferencia de Ficheros Peer-to-Peer (Sockets)

La transferencia de ficheros se realiza directamente entre los usuarios, sin intervención del servidor central. Para ello, el cliente mantiene una caché local con las direcciones IP y puertos de los usuarios conectados, obtenida mediante la operación USERS

Cuando un usuario invoca GETFILE, el programa cliente establece un socket TCP directo contra el hilo de escucha del usuario remoto que posee el fichero. El receptor abre el archivo localmente y transmite su contenido en bloques binarios a través del socket hasta alcanzar el fin de fichero

- Servicio Web (Python)

El conversor de mensajes es un microservicio web implementado en Python utilizando la librería Spyne para soportar el protocolo SOAP. Su única responsabilidad es normalizar el texto redactado por el usuario, eliminando los espacios en blanco repetidos para garantizar que las palabras estén separadas exclusivamente por un único espacio

El cliente de mensajería actúa como consumidor de este servicio (utilizando la librería Zeep), interceptando las operaciones SEND y SENDATTACH para procesar el texto con el servicio web local antes de enviarlo al servidor central

- Servicio de Registro de Operaciones (RPC)

Se ha implementado un servicio RPC en C que actúa como logger del sistema. Define una interfaz remota que recibe dos parámetros: el nombre del usuario y la operación ejecutada.

Cada vez que el Servidor Central procesa un comando (REGISTER, CONNECT, SEND, etc.), actúa como cliente RPC y realiza una llamada síncrona a este registrador. El registrador imprime por consola la traza de la operación. En el caso específico de SENDATTACH, también se adjunta e imprime el nombre del archivo asociado

## 4. Compilación y ejecución

Para la correcta compilación del sistema, se deben seguir estos pasos:

### Compilación:

- \$ make clean → limpia los binarios y archivos autogenerados
- \$ make → compila el servidor C, el RPC y ejecuta rpcgen

### Despliegue y Ejecución (4 terminales diferentes):

Terminal 1. Servicio de Registro (RPC):

```
./rpc_server
```

Terminal 2. Servicio Web SOAP (Conversor):

```
python3 src/ws_server.py
```

Terminal 3. Servidor Central:

```
export LOG_RPC_IP=127.0.0.1  
./server -p 8888
```

Terminal 4. Cliente:

```
python3 src/client.py -s localhost -p 8888
```

## 5. Batería de pruebas y resultados

A continuación, se detalla la una batería de casos de prueba diseñados y ejecutados para comprobar la robustez, fiabilidad y correcto funcionamiento de toda la arquitectura distribuida

El diseño de estas pruebas no se ha limitado a comprobar el flujo normal de ejecución (operaciones estándar de registro, conexión y mensajería en tiempo real ), sino que se ha puesto énfasis en la evaluación de casos extremos, la gestión de la concurrencia y la tolerancia a fallos del sistema. Se culmina con una validación en un entorno real con ordenadores diferentes conectados a la misma red local

## 1. REGISTRO Y BAJA

| ID  | Descripción                          | Terminal 1 (Carmen)                  | Terminal 2 (Ana) | Resultado Obtenido                                                  |
|-----|--------------------------------------|--------------------------------------|------------------|---------------------------------------------------------------------|
| 1.1 | registro exitoso de un nuevo usuario | REGISTER Carmen                      |                  | CLIENT 1: c> REGISTER OK<br>SERVER: s> REGISTER Carmen OK           |
| 1.2 | registro de un usuario duplicado     | REGISTER Carmen (ya ejecutado antes) | REGISTER Carmen  | CLIENT 2: c> USERNAME IN USE<br>SERVER: s> REGISTER Carmen FAIL     |
| 1.3 | baja exitosa                         | UNREGISTER Carmen                    |                  | CLIENT 1: c> UNREGISTER OK<br>SERVER: s> UNREGISTER Carmen OK       |
| 1.4 | baja de un usuario inexistente       |                                      | UNREGISTER Pepe  | CLIENT 2: c> USER DOES NOT EXIST<br>SERVER: s> UNREGISTER Pepe FAIL |

## 2. CONEXIÓN Y DESCONEXIÓN

| ID  | Descripción               | Terminal 1 (Carmen)               | Terminal 2 (Ana)            | Resultado Obtenido                                                                                                                                                                                          |
|-----|---------------------------|-----------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2.1 | conexión inicial exitosa  | REGISTER Carmen<br>CONNECT Carmen | REGISTER Ana<br>CONNECT Ana | CLIENT 1:<br>c> REGISTER OK<br>c> CONNECT OK<br><br>CLIENT 2:<br>c> REGISTER OK<br>c> CONNECT OK<br><br>SERVER:<br>s> REGISTER Carmen OK<br>s> CONNECT Carmen OK<br>s> REGISTER Ana OK<br>s> CONNECT Ana OK |
| 2.2 | control de doble conexión | CONNECT Carmen                    |                             | CLIENT 1:<br>c> USER ALREADY CONNECTED                                                                                                                                                                      |

|     |                             |  |                |                                                                                              |
|-----|-----------------------------|--|----------------|----------------------------------------------------------------------------------------------|
| 2.3 | desconexión exitosa         |  | DISCONNECT Ana | CLIENT 2:<br>c> DISCONNECT OK<br><br>SERVER:<br>s> DISCONNECT Ana OK                         |
| 2.4 | desconexión estando offline |  | DISCONNECT Ana | CLIENT 2:<br>c> DISCONNECT FAIL, USER NOT CONNECTED<br><br>SERVER:<br>s> DISCONNECT Ana FAIL |

### 3. USUARIOS

| ID  | Descripción                                          | Terminal 1 (Carmen) | Terminal 2 (Ana)            | Resultado Obtenido                                                                                                                                                                                |
|-----|------------------------------------------------------|---------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3.1 | listado de usuarios activos (terminal 2 antes que 1) |                     | CONNECT Ana<br><br>USERS    | CLIENT 2:<br>c> CONNECT OK<br>c> CONNECTED USERS (2 users connected) OK<br>Ana :: 127.0.0.1 :: 34207<br>Carmen :: 127.0.0.1 :: 39299<br><br>SERVER:<br>s> CONNECT Ana OK<br>s> CONNECTED USERS OK |
| 3.2 | listado estando offline                              |                     | DISCONNECT Ana<br><br>USERS | CLIENT 2:<br>c> DISCONNECT OK<br>c> CONNECTED USERS FAIL, USER IS NOT CONNECTED<br><br>SERVER:<br>s> DISCONNECT Ana OK                                                                            |

### 4. MENSAJERÍA

| ID  | Descripción                                        | Terminal 1 (Carmen)   | Terminal 2 (Ana) | Resultado Obtenido         |
|-----|----------------------------------------------------|-----------------------|------------------|----------------------------|
| 4.1 | envío a usuario conectado (terminal 2 antes que 1) | SEND Ana Hola que tal | CONNECT Ana      | CLIENT 2:<br>c> CONNECT OK |

|     |                                                       |                                    |                |                                                                                                                                                                                                                                         |
|-----|-------------------------------------------------------|------------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     |                                                       |                                    |                | <p>CLIENT 1:<br/>c&gt; SEND OK MESSAGE 1<br/>c&gt;<br/>c&gt; SEND MESSAGE 1 OK</p> <p>CLIENT 2:<br/>s&gt; MESSAGE 1 FROM Carmen Hola<br/>END</p> <p>SERVER:<br/>s&gt; SEND MESSAGE 1 FROM<br/>Carmen TO Ana</p>                         |
| 4.2 | envío a usuario desconectado (terminal 2 antes que 1) | SEND Ana Hola!!                    | DISCONNECT Ana | <p>CLIENT 2:<br/>c&gt; DISCONNECT OK</p> <p>CLIENT 1:<br/>c&gt; SEND OK MESSAGE 2</p> <p>SERVER:<br/>s&gt; MESSAGE 2 FROM Carmen TO Ana<br/>STORED</p>                                                                                  |
| 4.3 | recepción de pendientes                               |                                    | CONNECT Ana    | <p>CLIENT 2:<br/>c&gt; CONNECT OK<br/>c&gt;<br/>s&gt; MESSAGE 2 FROM Carmen Hola!!<br/>END</p> <p>SERVER:<br/>s&gt; SEND MESSAGE 2 FROM Carmen TO Ana</p>                                                                               |
| 4.4 | mensaje máximo (255 char)                             | SEND Ana<br><cadena_255_chars<br>> |                | <p>CLIENT 1:<br/>c&gt; SEND OK MESSAGE 1<br/>c&gt;<br/>c&gt; SEND MESSAGE 1 OK</p> <p>CLIENT 2:<br/>s&gt; MESSAGE 1 FROM CARMEN<br/>&lt;cadena_255_chars&gt;<br/>END</p> <p>SERVER:<br/>s&gt; SEND MESSAGE 1 FROM<br/>CARMEN TO ANA</p> |
| 4.5 | mensaje excede máx (300 char) (error)                 | SEND Ana<br><cadena_300_chars<br>> |                | <p>CLIENT 1:<br/>c&gt; SEND FAIL</p>                                                                                                                                                                                                    |

| 5. ADJUNTOS |                                                           |                                   |                  |                                                                                                                                                                                                                                                               |
|-------------|-----------------------------------------------------------|-----------------------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID          | Descripción                                               | Terminal 1 (Carmen)               | Terminal 2 (Ana) | Resultado Obtenido                                                                                                                                                                                                                                            |
| 5.1         | envío de adjunto (dest online)                            | SENDATTACH Ana /tmp/a.txt adjunto |                  | <p>CLIENT 1:<br/> c&gt; SENDATTACH OK MESSAGE 4<br/> c&gt;<br/> c&gt; SENDATTACH MESSAGE 4 adjunto OK</p> <p>CLIENT 2:<br/> c&gt; MESSAGE 4 FROM Carmen /tmp/a.txt<br/> END<br/> FILE adjunto</p> <p>SERVER:<br/> s&gt; SEND MESSAGE 4 FROM Carmen TO Ana</p> |
| 5.2         | envío de adjunto (dest. offline) (terminal 2 antes que 1) | SENDATTACH Ana /tmp/a.txt adjunto | DISCONNECT Ana   | <p>CLIENT 2:<br/> c&gt; DISCONNECT OK</p> <p>CLIENT 1:<br/> c&gt; SENDATTACH OK MESSAGE 5</p> <p>SERVER:<br/> s&gt; MESSAGE 5 FROM Carmen TO Ana<br/> STORED</p>                                                                                              |
| 5.3         | el usuario se vuelve a conectar                           |                                   | CONNECT Ana      | <p>CLIENT 2:<br/> c&gt; MESSAGE 5 FROM Carmen /tmp/a.txt<br/> END<br/> FILE adjunto</p> <p>CLIENT 1:<br/> c&gt; SENDATTACH MESSAGE 5 adjunto OK</p> <p>SERVER:<br/> s&gt; CONNECT Ana OK<br/> s&gt; SEND MESSAGE 5 FROM Carmen TO Ana</p>                     |



## 6. WEB SERVICE

| ID  | Descripción                         | Terminal 1 (Carmen)                 | Terminal 2 (Ana) | Resultado Obtenido                                                                                                                            |
|-----|-------------------------------------|-------------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 6.1 | tolerancia a fallos SOAP<br>(error) | SEND Ana prueba<br>(con WS apagado) |                  | Cliente 1 captura excepción, envía mensaje original. Servidor C lo procesa normalmente                                                        |
| 6.2 | funcionamiento web service          | SEND Ana hola<br>que tal ?          |                  | CLIENT 1:<br><br>c> SEND OK MESSAGE 1<br>c><br>c> SEND MESSAGE 1 OK<br><br>CLIENT 2:<br><br>s> MESSAGE 1 FROM Carmen<br>hola que tal ?<br>END |

## 7. FICHEROS P2P

| ID  | Descripción                         | Terminal 1 (Carmen)                                 | Terminal 2 (Ana)                     | Resultado Obtenido                                                                                                                                                                                                                                                       |
|-----|-------------------------------------|-----------------------------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7.1 | descarga P2P                        | CONNECT Carmen                                      | GETFILE Carmen<br>/tmp/a.txt ./b.txt | CLIENT 2:<br>c> GETFILE OK. Archivo guardado como ./b.txt                                                                                                                                                                                                                |
| 7.2 | getfile de usuario<br>(error P2P)   | DISCONNECT Carmen                                   | GETFILE Carmen<br>/tmp/a.txt ./b.txt | CLIENT 1:<br>c> DISCONNECT OK<br>CLIENT 2:<br>c> FILE TRANSFER FAILED, user not connected                                                                                                                                                                                |
| 7.3 | fichero remoto no existe<br>(error) | CONNECT Carmen<br><br>(no tiene falso.txt en su PC) | GETFILE Carmen<br>falso.txt ./b.txt  | La conexión P2P se establece. El hilo de Carmen intenta abrir el archivo, pero al no existir, bloquea el fallo y cierra el socket sin enviar datos. Ana genera un archivo ./b.txt local vacío (0 bytes)<br><br>CLIENT 2:<br>c> GETFILE OK. Archivo guardado como ./b.txt |

## 8. RPC LOGGER

| ID  | Descripción                | Terminal 1 (Carmen)                            | Terminal 2 (Ana) | Resultado Obtenido                                                                                        |
|-----|----------------------------|------------------------------------------------|------------------|-----------------------------------------------------------------------------------------------------------|
| 8.1 | registro de operacion SEND | SEND Ana Hola                                  |                  | SERVIDOR RPC:<br>Carmen<br>SEND                                                                           |
| 8.2 | registro de SENDATTACH     | SENDATTACH Ana /tmp/x.txt adjunto              |                  | SERVIDOR RPC:<br>Carmen<br>SENDATTACH adjunto                                                             |
| 8.3 | RPC caído                  | REGISTER Carmen (el servidor RPC está apagado) |                  | CLIENT 1:<br>c> REGISTER OK<br><br>SERVER:<br>s> 127.0.0.1: RPC: Remote system error - Connection refused |

## 9. COMUNICACIÓN

| ID  | Descripción                                                | Terminal 1 (Carmen)                                    | Terminal 2 (Ana)                                       | Resultado Obtenido                                                                                                                                                                           |
|-----|------------------------------------------------------------|--------------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9.1 | puerto de servidor incorrecto                              | python3 client.py -p 9999<br><br>REGISTER Carmen       |                                                        | CLIENT 1:<br>c> REGISTER FAIL<br>Motivo real del fallo: [Erno 111]<br>Connection refused                                                                                                     |
| 9.2 | cliente cierra de golpe (ctrl + C en el cliente conectado) | CONNECT Carmen<br><br>Ctrl + C                         | SEND Carmen Hola                                       | CLIENT 1: (desconectado)<br><br>CLIENT 2:<br>c> SEND OK MESSAGE 1<br><br>SERVER:<br>s> MESSAGE 1 FROM Ana TO Carmen<br>STORED                                                                |
| 9.3 | múltiples clientes (conurrencia)                           | Envío simultáneo (script) de múltiples peticiones SEND | Envío simultáneo (script) de múltiples peticiones SEND | El servidor C despliega un hilo para cada petición. Gracias al pthread_mutex_lock, la lista global de usuarios no sufre corrupción de memoria y todos los mensajes se procesan correctamente |

## 10. PROTOCOLO Y TOLERANCIA A FALLOS

| ID   | Descripción                     | Terminal 1 (Carmen)                     | Terminal 2 (Ana)                          | Resultado Obtenido                                                                                                            |
|------|---------------------------------|-----------------------------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 10.1 | operacion no valida             | INVENTO Carmen                          |                                           | CLIENT 1:<br>Error: command INVENTO not valid.                                                                                |
| 10.2 | falta de parámetros             | SENDATTACH Ana                          |                                           | CLIENT 1:<br>Syntax error. Usage: SENDATTACH <userName> <message> <fileName>                                                  |
| 10.3 | operaciones con una IP distinta | REGISTER Carmen (desde IP 192.168.1.10) | DISCONNECT Carmen (desde IP 192.168.1.20) | El servidor verifica que un usuario solo se puede desconectar si la operación proviene de la IP desde la que se ha registrado |

## PRUEBA DEFINITIVA

### ENVÍO DE ARCHIVOS ENTRE 2 ORDENADORES DISTINTOS

Objetivo: Verificar el correcto funcionamiento de la transferencia de ficheros (P2P) entre dos ordenadores físicos diferentes conectados a la misma red (WiFi). El servidor central se encuentra en ejecución en la IP asignada (<IP\_ASIGNADA>) y puerto 8888

#### 1. Conexión al servidor

- Ordenador 1 (ANA): Inicia el cliente conectándose a la IP del servidor  

```
$ python3 src/client.py -s <IP_ASIGNADA> -p 8888
c> REGISTER ANA
c> CONNECT ANA
```
- Ordenador 2 (MIGUEL): Inicia el cliente en su máquina conectándose al mismo servidor  

```
$ python3 src/client.py -s <IP_ASIGNADA> -p 8888
c> REGISTER MIGUEL
c> CONNECT MIGUEL
```

#### 2. Envío del mensaje con fichero adjunto

- Ordenador 1 (ANA): Ejecuta el comando SENDATTACH indicando como destinatario a Miguel, un mensaje de texto ("1") y la ruta absoluta del fichero a enviar

```
c> SENDATTACH MIGUEL 1 /home/ana-sanz/Escritorio/DISTRIBUIDOS/prueba.txt
```

- Salida ordenador 1: El sistema confirma la correcta entrega del mensaje y del adjunto al destinatario

```
c> SENDATTACH MESSAGE 1 /home/ana-sanz/Escritorio/DISTRIBUIDOS/prueba.txt OK
```

### 3. Recepción y descarga del fichero

- Ordenador 2 (MIGUEL): Recibe la notificación de que tiene un archivo disponible y procede a descargarlo en su directorio local mediante el comando GETFILE

```
c> GETFILE ANA /home/ana-sanz/Escritorio/DISTRIBUIDOS/prueba.txt  
/home/miguel-merino/Escritorio/hola.txt
```

- Salida en el ordenador 2: El cliente confirma que la transferencia directa mediante Sockets ha finalizado con éxito

```
c> GETFILE OK. Archivo guardado como /home/miguel-merino/Escritorio/hola.txt
```

Tanto la estructura del archivo como el contenido del mismo se transfieren correctamente entre ambos ordenadores

## 6. Conclusiones y problemas encontrados

Durante esta práctica hemos asimilado la convergencia de diferentes tecnologías y protocolos explicados a lo largo de la asignatura (C, Python, Sockets TCP, Web Service y RPC) dentro de una arquitectura distribuida completamente funcional.

El principal desafío ha consistido en garantizar la seguridad en la concurrencia del servidor central. Para lograrlo, se han usado mecanismos de exclusión mutua (mutex) orientados a proteger las estructuras de datos compartidas (lista de usuarios), previniendo condiciones de carrera.

Durante la fase de integración, encontramos dos incidencias con la compatibilidad del entorno

1. Incompatibilidad del servicio web con python 3.12 (biblioteca spyne)
2. Gestión de dependencias RPC en Linux

En conclusión, el desarrollo de este proyecto ha afianzado nuestra comprensión sobre la construcción de sistemas tolerantes a fallos